



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计期末实验报告

ANN 选题并行优化期末报告

姓名：申健强

学号：2313119

专业：计算机科学与技术

指导教师：王刚

2025 年 7 月 6 日

目录

一、 前言	1
(一) 实验环境	1
(二) 实验内容概述	1
1. ANN 算法背景	1
2. ANNS 的优化方向	2
(三) 实验代码	3
二、 不同算法的优化原理	3
(一) PQ 类量化方法	3
1. SQ (Scalar Quantization)	3
2. PQ (Product Quantization)	3
3. OPQ (Optimized Product Quantization)	4
4. 重排序方法 (PQ + rerank)	4
(二) IVF 倒排索引	5
(三) HNSW 可导航小世界	5
(四) kd-tree 算法	6
1. 构建过程	6
2. 精确 NN 查询	6
3. 近似控制与加速 ANN	7
(五) LSH 算法	7
1. LSH 哈希族定义	7
2. 碰撞概率与哈希增强	8
3. 典型哈希族示例	8
4. 查询流程与复杂度	8
(六) 混合方法	9
1. IVF+PQ	9
2. IVF+HNSW 混合索引	9
3. IVF 算法的综合并行尝试	10
三、 不同并行方法如何加速计算	11
(一) SIMD	11
(二) OpenMP/pthread 多线程	11
(三) MPI 多进程	11
(四) GPU 加速计算	12
四、 实验设计探究	12
(一) PQ 类方法 +SIMD	12
(二) IVF/HNSW 类方法 + 多线程	12
(三) 多进程优化算法	12
(四) GPU 加速查询间并行	13
(五) 探究: kdtree 算法与 LSH 算法并行化尝试	13
(六) 探究: 杂交方法效果	13
(七) 探究: 混合并行尝试	13

五、 实验结果可视化分析	13
(一) PQ 类方法 +SIMD	13
1. 实验结果	13
2. 召回率与延迟的权衡分析	13
3. SIMD 指令集加速效果	14
4. 子空间数量对 PQ 和 OPQ 性能的影响	14
5. 重排序因子对性能的影响	15
(二) IVF/HNSW 类方法 + 多线程	16
1. 朴素 IVF 算法多线程	16
2. HNSW 算法及其多线程	16
(三) 多进程优化算法	16
1. 分片 HNSW 算法	16
2. 进程数对实验结果的影响分析	17
(四) GPU 加速查询并行	19
1. GPU 批次加速方案加速比	19
2. CPU 选择 TOP-K 与 GPU 实现 TOP-K 加速对比	20
3. 不同分组方式对比	21
(五) 探究: kdtree 算法与 LSH 算法并行化尝试	22
1. 参数分析	23
(六) 探究: 杂交方法效果	24
1. PQ 与 IVF 混合的两种方法对比分析	24
2. HNSW 与 IVF 算法混合方法分析	24
(七) 探究: 混合并行尝试	26
1. HNSW 算法混合并行尝试	26
2. 流水线并行结果分析	27
六、 综合分析	29
七、 总结	29

一、前言

(一) 实验环境

实验环境：

- 本地 CPU: Intel(R) Core(TM) Ultra 5 125H 1.20 GHz
- 线上测试: OpenEuler 服务器, GPU: RTX3090(24GB 显存) 显卡服务器
- IDE: Vscode
- 编译器: gcc11.4.0, mpicxx
- 多线程: openmp/pthread
- 操作系统: 本地测试除 mpi 之外均为 Windows11, 线上测试使用 Linux, 本地 MPI 实验使用 WSL
- simd 指令集: 本地: SSE/AVX, 线上: Neon

(二) 实验内容概述

1. ANN 算法背景

背景: 最近邻搜索 (Nearest Neighbor Search, NNS) 旨在在高维空间中找到与查询向量最相似的向量。由于深度学习技术可以将图像、文本、音频等非结构化数据转换为高维向量, 并保持语义相似性 (例如相似图片的向量距离较近), NNS 在推荐系统、图像检索和大语言模型等领域具有广泛应用。然而, 精确 NNS 在高维大规模数据集上的计算复杂度极高 (例如, 数据集规模 $N > 10^6$, 维度 $d > 100$), 使得其在实际场景中难以应用。因此, 近似最近邻搜索 (ANNS) 成为更实用的替代方案, 旨在在保持较高准确度的同时显著降低计算开销。

问题定义:

- 给定数据集 $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$ 。
- 对于查询向量 $\mathbf{q} \in \mathbb{R}^d$, ANNS 的目标是找到集合 $S \subseteq X$, 使得 S 中的向量与 $\mathbf{q}$ 足够接近, 并满足一定的准确度要求。

准确度衡量: 常用召回率 (recall) 评估 ANNS 的准确性:

$$\text{recall}@k = \frac{|KNN(\mathbf{q}) \cap KANN(\mathbf{q})|}{k}$$

其中 $KNN(\mathbf{q})$ 是精确的 k 个最近邻集合, $KANN(\mathbf{q})$ 是 ANNS 返回的 k 个近似最近邻集合。本实验中, 我们关注在达到特定召回率 (0.9) 时的算法延迟。

具体优化技术:

- 硬件层面:
 - SIMD 指令并行
 - openmp/pthread 多线程
 - MPI 多进程
 - GPU 运算加速

• 算法层面：

- 基于数据压缩：SQ, PQ, OPQ
- 基于索引构建：kd-tree、LSH, IVF, HNSW

本次实验基于 DEEP100K 数据集（维度为 96 的图像向量），重点探索上述优化技术在 ANN 中的实现与性能表现。

2. ANNS 的优化方向

ANNS 的优化主要集中在以下两个方面：

1. 加速距离计算：

- 使用 SIMD 指令并行化距离计算，提升单次计算的效率。
- 采用量化技术（如 PQ、OPQ、SQ）减少浮点运算量，降低计算复杂度。

2. 减少访问的数据点数量：

- 构建高效索引结构（如 HNSW、IVF）以快速定位候选集，减少需要计算的向量数量。
- 使用混合方法（如 PQ + rerank）结合快速筛选和精确重排序，在保证准确度的同时优化性能。

除了上述两个算法层面的优化之外，我们还能利用 GPU 强大的吞吐能力对 ANN 作查询间的并行优化，对大量的查询进行批次优化。

事实上，在 ANN 算法中存在一个由速度、准确性和内存构成的不可能三角，也就是说我们不可能构成一个速度又快，准确性高且索引内存小的 ANN 算法，在并行程序设计课程中我们一般不考虑索引大小，更注重速度和准确性的提升。



图 1: ANN 算法不可能三角

另一个值得探讨的问题在于，根据构建索引方式的不同，我们对 ANN 算法可以分为以下几类：

- 以树为索引的算法：KD-tree 算法
- 以 Hash 构建索引的算法：LSH 局部敏感哈希算法
- 以聚类方式构建索引的算法：IVF 倒排索引算法

- **以图构建索引的算法：**HNSW 层次化可导航小世界算法

在本次期末报告中，我们也将深入探讨不同构建方法的优缺点和使用场景。

(三) 实验代码

与前几次实验相同，本人实验的所有代码，实验结果，包括但不限于可视化和算法脚本都可以在https://github.com/sjq0098/Parallel_Programs.git中查看。

二、 不同算法的优化原理

在我们之前的实现中我们实现了不同的算法及其并行优化，在此处我们更为体系化的实现概述，我们加上 LSH 与 kdtree 算法及其并行化进行优化尝试。

(一) PQ 类量化方法

PQ 类量化方法是一类数据压缩技术，通过将浮点数转换为整数从而减少数据的运算量，具体而言，我们可以将它们分为以下几类。

1. SQ (Scalar Quantization)

原理：SQ (标量量化) 将每个维度的浮点数值映射为低精度整数 (例如 8 位无符号整数)，从而减少存储需求并加速计算。

数学推导：

- 对于每个维度 i ，将 x_i 映射到 $[0, 255]$ 范围内：

$$x'_i = \text{round} \left(255 \cdot \frac{x_i - \min_i}{\max_i - \min_i} \right)$$

其中 $\min_i$ 和 $\max_i$ 是第 i 维的最小值和最大值。

距离计算：使用量化后的整数值近似计算距离，结合 SIMD 指令 (如处理 16 个 8 位整数的并行运算) 进一步提升效率。

优势：SQ 实现简单，存储开销低，且易于与 SIMD 结合加速计算。

2. PQ (Product Quantization)

原理：PQ (乘积量化) 通过将高维向量分解为多个低维子空间并分别量化，通过距离查表 LUT 降低存储和计算开销，同时保持近似距离计算的准确性。

距离计算：对于查询向量 $\mathbf{q}$ ，预先计算其子向量与所有聚类中心的距离，存储在查找表 LUT 中。在搜索时，通过查找表快速计算量化向量与 $\mathbf{q}$ 的近似距离。

优势：PQ 将存储需求从原始浮点向量减少到量化索引，同时通过查找表加速距离计算。

缺点：PQ 采用近似的方式实现距离的计算优化，但近似计算的也有降低精度的代价，这个代价我们将在混合方法中给出一定改善。

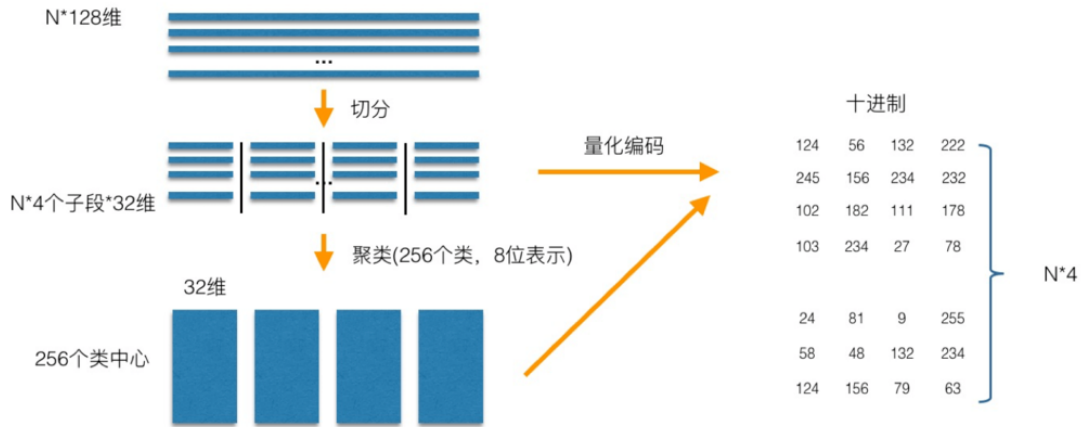


图 2: PQ 原理展示图

3. OPQ (Optimized Product Quantization)

原理: OPQ 是 PQ 的改进版本, 通过引入旋转矩阵优化子空间划分, 减少量化误差, 提高搜索准确性。

数学推导:

- 引入正交旋转矩阵 $\mathbf{R}$, 将原始向量 $\mathbf{x}$ 变换为 $\mathbf{x}' = \mathbf{R}\mathbf{x}$ 。
- 对变换后的向量 $\mathbf{x}'$ 应用 PQ 量化。
- 优化目标是最小化量化误差:

$$\min_{\mathbf{R}, \mathbf{C}} \sum_{\mathbf{x}} \|\mathbf{x} - \mathbf{R}^{-1}\hat{\mathbf{x}}'\|^2$$

其中 $\hat{\mathbf{x}}'$ 是 $\mathbf{x}'$ 的量化表示, $\mathbf{C}$ 是聚类中心集合。

实现: 通过迭代优化 $\mathbf{R}$ 和 $\mathbf{C}$, OPQ 在保持 PQ 高效性的同时, 进一步提升量化精度。

缺点: OPQ 能够在保留 PQ 的高效性的同时提升量化精度, 但缺点在于索引构建成本过高。

4. 重排序方法 (PQ + rerank)

对于大部分的 PQ 算法而言, 其精度降低太快, 无法正常进行使用, 而另一个更合理的方案是我们使用 PQ 算法来确定大致范围, 再启动精确重排序, 这样才能再可使用范围内实现一个更好的精度与速度的平衡。

原理: 混合方法结合 PQ 的快速近似搜索和 rerank 的精确重排序, 先通过 PQ 筛选候选集, 再对候选集进行精确计算以提高准确性。

数学推导:

- **PQ 搜索:** 利用 PQ 量化向量快速计算近似距离, 生成候选集 S' 。
- **rerank:** 对 S' 中的向量使用原始向量计算距离, 排序后输出最终结果。

优势: 该方法在保证高召回率的同时, 通过 rerank 提升了搜索精度, 适用于对准确性要求较高的场景。

(二) IVF 倒排索引

• 算法简介:

- 首先对全部向量集 $X = \{x_i\}_{i=1}^N \subset \mathbb{R}^D$ 进行 K 均值聚类, 得到质心集合 $\{c_j\}_{j=1}^K$;
- 对于查询向量 $\mathbf{q} \in \mathbb{R}^D$, 计算其与每个质心的内积:

$$s_j = \langle \mathbf{q}, c_j \rangle, \quad j = 1, \dots, K,$$

选择内积值最大的 n_{probe} 个簇:

$$\mathcal{S} = \operatorname{argmax}_{|\mathcal{S}|=n_{\text{probe}}} \sum_{j \in \mathcal{S}} s_j.$$

仅在这些簇的倒排列表中, 继续计算候选点 x_i 与 $\mathbf{q}$ 的内积:

$$\langle \mathbf{q}, x_i \rangle,$$

并从中选取最大 Top- k 。

- **设计动机:** 通过粗粒度的聚类将搜索空间划分为多个区域, 仅在最相关的簇中进行精细搜索, 既能显著减少每次查询的候选向量数目, 又能兼顾较高的检索准确率。
- **性能特征:** 预先的 k -means 聚类成本较高 ($O(NKD)$), 但只需离线执行一次; 由于我们目前的方法都只考虑查询的时间, 其实离线造成的成本可以忽略不计。

(三) HNSW 可导航小世界

• 算法简介:

- 构建多层图结构, 每层节点拥有若干邻居;
- 查询从顶层节点 v_0 出发, 逐层以最大内积为目标进行贪心跳跃:

$$v \leftarrow \arg \max_{u \in N_i(v)} \langle \mathbf{q}, u \rangle,$$

重复直到该层无法进一步提升内积, 再进入下一层;

- 底层使用 beam search, 维护候选集 $\mathcal{C}$ 和结果集 $\mathcal{R}$, 不断从 $\mathcal{C}$ 中取内积最大的点扩展其邻居并更新。
- **设计动机:** 借鉴小世界网络的“短路径”性质, 通过多层次图结构在高层快速锁定大致位置, 在底层高精度搜寻, 大幅减少全图搜索的跳跃次数与候选扩展量。
- **复杂度分析:** 在合适的参数 (efConstruction, efSearch) 下, 平均查询复杂度近似为 $O(\log N)$; 实际性能在高维度与不同数据分布下都表现稳定。
- **并行化切点:**

1. **查询阶段:** 虽然搜索顺序受限, 但在每轮候选扩展时, 可并行对候选邻居 u 计算 $\langle \mathbf{q}, u \rangle$;

下图3展示了 hnsw 的查询过程¹

¹图片引用自: <https://zhuanlan.zhihu.com/p/397628955>

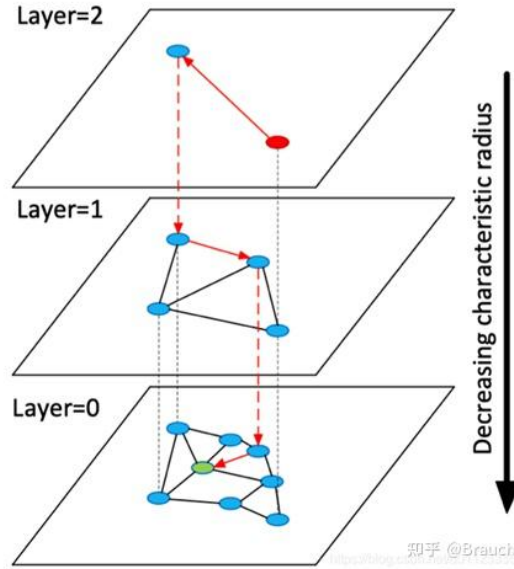


图 3: hnsw 查询过程

(四) kd-tree 算法

kd-tree (k -维二叉树) 是一种用于加速最近邻搜索 (Nearest Neighbor, NN) 和近似最近邻搜索 (Approximate Nearest Neighbor, ANN) 的空间划分数据结构。但需要注意的是, 对于 kdtree 而言, 我们更多的是使用 L2 距离对其进行计算和优化, 对于内积距离, 其一般实用程度不高。

1. 构建过程

给定数据集

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^k,$$

kd-tree 的构建采用递归二分法。假设当前节点要处理的点集为 X' , 深度为 d , 则:

1. 选择分割维度:

$$j = d \bmod k.$$

2. 在第 j 维上对 X' 按中位数进行排序, 取中位数点为 $\mathbf{x}_{\text{med}}$:

$$\mathbf{x}_{\text{med}} = \text{median}\{x^{(j)} : \mathbf{x} \in X'\}.$$

3. 创建节点存储 $\mathbf{x}_{\text{med}}$, 并将 $X' \setminus \{\mathbf{x}_{\text{med}}\}$ 分为左右子集:

$$X_{\text{left}} = \{\mathbf{x} \in X' : x^{(j)} < x_{\text{med}}^{(j)}\}, \quad X_{\text{right}} = \{\mathbf{x} \in X' : x^{(j)} \geq x_{\text{med}}^{(j)}\}.$$

4. 递归构建左右子树, 直到 $|X'| \leq 1$ 或达到预设深度为止。

该过程的时间复杂度平均为 $O(n \log n)$ 。

2. 精确 NN 查询

给定查询点 $\mathbf{q} \in \mathbb{R}^k$, kd-tree 的精确最近邻搜索采用深度优先遍历并结合“有界剪枝”策略, 过程可描述如下:

首先从根节点开始，对当前节点（存储点 $\mathbf{x}_{\text{node}}$ 、深度 d ）计算分割维度 $j = d \bmod k$ 。比较 $q^{(j)}$ 与 $x_{\text{node}}^{(j)}$ ，优先沿着包含查询点的一侧子树（左或右）递归下探。一旦到达叶节点，便用欧氏距离更新当前的最小距离上界

$$r \leftarrow \min(r, \|\mathbf{q} - \mathbf{x}_{\text{leaf}}\|_2).$$

回溯时，每当返回到父节点，就判断查询点到该节点超平面的距离 $|q^{(j)} - x_{\text{node}}^{(j)}|$ 是否小于当前距离上界 r 。若满足

$$|q^{(j)} - x_{\text{node}}^{(j)}| < r,$$

则该超平面对面子树中可能存在距离更小的点，需继续递归搜索；否则可剪去这一分支，无需访问，从而减少不必要的距离计算。如此，算法利用空间划分构建的有界条件，在平均情况下仅沿着少数路径和必要的回退分支进行搜索，使得查询成本接近 $O(\log n)$ ，而在最坏情况下仍保证正确性但退化至 $O(n)$ 。“

3. 近似控制与加速 ANN

为了在大规模数据上获得更快速度，可在搜索过程中引入近似策略。常用方法如下：

- **限制回溯次数**：设置参数 L ，即最多允许访问的叶节点数。当访问叶节点数达到 L 时即刻返回当前最优解 $\hat{\mathbf{x}}$ 。这样可将查询复杂度从 $O(\log n)$ 进一步控制在 $O(L)$ ：

$$\text{访问叶节点数} \leq L \implies \text{时间复杂度 } O(L).$$

- **误差上界 ε -近似**：允许结果满足

$$\|\mathbf{q} - \hat{\mathbf{x}}\| \leq (1 + \varepsilon) \|\mathbf{q} - \mathbf{x}^*\|,$$

通过在回溯判别条件中放宽：

$$|q^{(j)} - x_{\text{node}}^{(j)}| < (1 + \varepsilon) r,$$

可显著减少回溯次数。

- **多棵树合并 (Forest)**：构建 T 棵随机平衡的 kd-tree，对每棵树分别进行近似搜索，然后取最优结果，进一步提升召回率与速度的折中效果。

这样，通过选择合适的 (L, ε, T) 参数，可在精度与查询速度之间实现灵活平衡，从而大幅加速 ANN 检索而仅带来可控的近似误差。

（五） LSH 算法

局部敏感哈希 (Locality-Sensitive Hashing, LSH) 是一类通过哈希函数族将相似对象映射到相同桶 (bucket) 中，从而以次线性时间近似解决高维空间中的最近邻检索问题的方法。

1. LSH 哈希族定义

对于给定的距离度量 $D(\cdot, \cdot)$ ，一族哈希函数 $\mathcal{H} = \{h : \mathbb{R}^k \rightarrow \mathcal{U}\}$ 称为 (r, cr, p_1, p_2) -局部敏感，当且仅当对任意 $\mathbf{x}, \mathbf{y} \in \mathbb{R}^k$ 有：

$$\begin{cases} D(\mathbf{x}, \mathbf{y}) \leq r, & \Pr_{h \in \mathcal{H}}[h(\mathbf{x}) = h(\mathbf{y})] \geq p_1, \\ D(\mathbf{x}, \mathbf{y}) \geq cr, & \Pr_{h \in \mathcal{H}}[h(\mathbf{x}) = h(\mathbf{y})] \leq p_2, \end{cases}$$

其中 $c > 1$ ，且 $p_1 > p_2$ 。这样，近距离点更可能落到同一哈希桶中，而远距离点落入同一桶的概率较低。

2. 碰撞概率与哈希增强

为了调节检索的召回率与精度, LSH 通常采用以下两步“增强”策略:

- **组合 (Concatenation)**: 将 k 个独立哈希函数拼接组成一个复合哈希函数

$$g(\mathbf{x}) = (h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_k(\mathbf{x})).$$

则对于任意两点 $\mathbf{x}, \mathbf{y}$,

$$\Pr[g(\mathbf{x}) = g(\mathbf{y})] = (\Pr[h(\mathbf{x}) = h(\mathbf{y})])^k.$$

通过增大 k 可以降低远点误碰撞概率。

- **多表 (Amplification via Tables)**: 构建 L 个独立的 g 函数, 形成 L 张哈希表。查询时, 只需在任意一张表中命中即可:

$$P_{\text{hit}}(\mathbf{x}, \mathbf{y}) = 1 - (1 - p_1^k)^L.$$

通过增大 L 可以提升近点的命中概率。

3. 典型哈希族示例

- **欧氏空间中的 p -稳定分布哈希**: 令 $\mathbf{a} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $b \sim \text{Unif}[0, w]$, 定义

$$h_{\mathbf{a}, b}(\mathbf{x}) = \left\lfloor \frac{\mathbf{a}^\top \mathbf{x} + b}{w} \right\rfloor.$$

对于距离度量 $D(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2$, 该哈希族满足 2-稳定分布的局部敏感性。

- **海明空间中的位采样哈希**: 对 $\{0, 1\}^k$ 中的向量, 随机选择坐标索引 i , 定义

$$h_i(\mathbf{x}) = x_i.$$

对于海明距离 $D(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_0$, 这一哈希族满足局部敏感性。

4. 查询流程与复杂度

给定查询向量 $\mathbf{q}$, LSH 查询步骤如下:

1. 对 $\mathbf{q}$ 计算 L 个复合哈希值 $\{g_\ell(\mathbf{q})\}_{\ell=1}^L$ 。
2. 在每张哈希表 ℓ 的桶 $g_\ell(\mathbf{q})$ 中收集候选集 $\mathcal{C}_\ell$ 。
3. 合并所有候选点 $\mathcal{C} = \bigcup_{\ell=1}^L \mathcal{C}_\ell$, 对每个候选点计算真实距离, 取最小者作为近似解。

在适当选择 (k, L) 后, LSH 的查询时间期望为

$$O(L \cdot T_h + |\mathcal{C}|),$$

其中 T_h 为单次哈希计算时间, 通常远小于线性扫描的 $O(n)$, 且 $|\mathcal{C}| \ll n$ 。通过控制 k 与 L , 可以在检索速度与近似精度之间进行平衡, 从而高效地加速 ANN 检索。

(六) 混合方法

1. IVF+PQ

• PQ → IVF (先 PQ, 后 IVF)

1. **全局 PQ 码本训练**: 在全集上将 d 维空间等分为 M 个子空间, 对每个子空间使用 K -means++ 训练全局质心, 共得到 $M \times K$ 个中心点, 形成一套共享的 PQ 码本。
2. **IVF 聚类指派**: 仍以原始浮点向量为输入, 使用 k -means++ 在全集上聚类得到 n_{list} 个粗簇中心, 建立倒排文件 (Inverted File)。
3. **数据写入**: 每个向量既记录其所属簇 ID, 又计算并存储其 M 字节的全局 PQ 码。
4. **查询阶段**

- (a) **粗排 (IVF)**: 对查询向量 q 与所有簇中心 c_i 计算标准化内积距离

$$d_{\text{coarse}}(q, c_i) = 1 - \langle q, c_i \rangle,$$

并选出前 n_{probe} 个簇。

- (b) **距离表生成**: 仅对 q 计算一次全局 PQ 的 $M \times K$ 子空间距离表。
- (c) **精排 (PQ)**: 遍历所选簇的倒排文件, 对每条 PQ 码查表累加 M 段子距离, 维护一个大小为 ef 的小顶堆, 保留候选。

• IVF → PQ (先 IVF, 后 PQ)

1. **IVF 聚类指派**: 同上, 对全集向量聚成 n_{list} 个粗簇, 并建立倒排文件。
2. **簇内 PQ 码本训练**: 在每个簇内, 仍将高维向量拆分为 M 个子空间, 并对每个子空间独立运行 K -means++, 得到 $M \times K$ 个本地质心, 形成该簇专属的 PQ 码本。
3. **数据写入**: 每个簇内向量使用其对应簇的局部 PQ 码本计算并存储 PQ 码。
4. **查询阶段**
 - (a) 粗排同上, 选出 n_{probe} 个簇。
 - (b) 对于每个选中簇, 使用该簇的本地 PQ 码本生成子空间距离表 (共 n_{probe} 次)。
 - (c) 在各簇倒排文件中查表累加子距离, 合并所有簇的候选至大小为 ef 的堆。
 - (d) 对堆中前 k 个候选做一次原始浮点向量的精确内积重排, 以提高 Top- k 准确率。

2. IVF+HNSW 混合索引

在该方案中, 我们将图索引 HNSW 嵌入到 IVF 的“精排”阶段, 以提升检索精度和速度:

1. **IVF 构建**: 对全集先执行 n_{list} 聚类并建立倒排列表。
2. **簇内 HNSW 构建**: 对每个倒排簇中包含的向量单独构建一座 HNSW 图, 图节点表示向量, 图层与长边保证高效跳跃搜索。
3. **MPI 跨簇并行**: 查询时, 主进程将 n_{probe} 个簇 ID 分发给不同 MPI 进程, 各进程并行执行 HNSW 搜索。
4. **多线程簇内并行**: 若单个 HNSW 查询实现多线程 (如多起点初始化、并行邻居探测), 则在每个 MPI 进程内部进一步利用多核加速。

5. **结果合并**: 各进程返回局部 Top- k 候选, 主进程收集并在全局堆中合并, 必要时再做精确重排。

我们绘制如下过程图4来表示算法的大概过程:²:

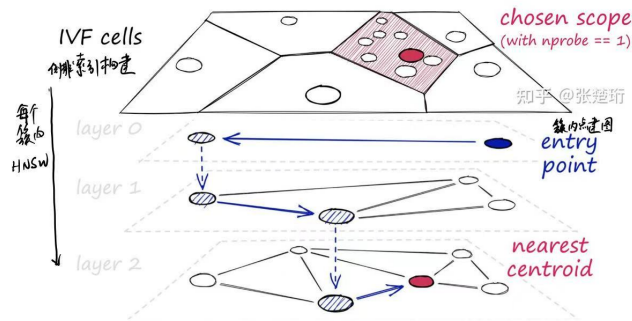


图 4: IVF+HNSW 算法流程示意

3. IVF 算法的综合并行尝试

启发自计算机组成原理课程的流水线设计, 针对纯 IVF 框架, 我们设计了如下多级并行与硬件加速方案:

1. **MPI 簇间并行**: 将 n_{list} 个倒排簇等分到不同进程/节点, 实现粗排和倒排查询的分布式执行。
2. **多线程簇内并行**: 每个进程内部使用线程池, 按批次并行读取倒排列表、解码 PQ、累加距离与堆更新。
3. **SIMD 加速距离计算**: 在向量与查询向量的内积距离计算中, 利用 AVX/NEON 指令集进行数据对齐与批量计算, 提升单核计算带宽。
4. **流水线并行**:
5. **流水线并行与非阻塞汇总**: 整体分为“粗排 → 异步簇内 → 本地收集 → 非阻塞 MPI 汇总”四个阶段。
 - **阶段 1-> 启动阶段 2**: 一旦粗排确定前 n_{probe} 簇, 即刻触发簇内搜索, 无需等候全部粗排完成。
 - **阶段 2 与阶段 3 并行**: 异步线程写就 `AsyncSearchResult.ready` 标志后, 主线程立即开始收集候选, 不阻塞其它线程继续计算。
 - **阶段 3 与阶段 4 重叠**: 在轮询收集本地 (score, id) 时, 主线程可以同时发起并驱动非阻塞 MPI 全局汇总。
 - **阶段 4 完成后清洗排序**: 在 MPI 汇总数据到达之前, 其他候选收集仍可继续, 最大化通信—计算重叠率, 最后统一去重并全局 Top- k 排序。

²原图来自: <https://zhuanlan.zhihu.com/p/595249861>, 我们对其进行了改造来表述我们需要的算法过程

三、 不同并行方法如何加速计算

在本学习的学习中，我们学习了不同的并行优化工具，其概览如下，我们将活用以下工具对 ANN 进行优化。

(一) SIMD

SIMD (Single Instruction, Multiple Data) 是指在一条指令周期内对多个数据元素并行执行相同操作的机制。处理器通过向量寄存器（如 AVX、NEON 等）一次性加载多个浮点或整型数值，并在硬件级别对它们执行加/减/乘/除等运算。由于指令发射次数大幅减少，SIMD 能在数据高度同质且计算密集的场景（如矩阵乘法、卷积等）中获得 w 倍（ w 为向量宽度）左右的加速。

(二) OpenMP/pthread 多线程

OpenMP 和 pthread 都是在共享内存系统上实现多线程并行的编程接口：

- **OpenMP**：通过编译指令（`#pragma omp`）和运行时库自动将循环或代码块分解到多个线程，负责负载均衡和线程同步。适合对现有串行代码进行低侵入性并行化。
- **pthread**：提供 POSIX 线程 API，程序员手动创建、调度、同步线程，灵活性更高但编程复杂度也更大。

pthread 线程池理论 在线程池（thread pool）模型中，预先创建 T 个持久工作线程（workers），并维护一个任务队列（task queue）。当新的并行任务到来时，将任务封装为独立的工作单元并推入队列，空闲线程从队列中不断取出任务并执行。线程池的优越性体现在：

- **线程重用**：避免每次任务都创建/销毁线程的高开销，将线程创建成本摊薄到多次任务执行中；
- **并发控制**：通过固定线程数 T 限制并发度，防止因过多线程导致的上下文切换和资源争夺；
- **动态负载均衡**：工作线程从共享队列中“拉取”任务，可天然实现任务均衡，而无需手动分配任务子集；
- **响应式扩展**：可在运行时根据系统负载或任务特性调整线程池大小，平衡吞吐率与延迟；
- **缓存友好**：持久线程维护自己的工作栈和局部缓存（cache），有助于利用数据局部性，提升整体性能。

在多核 CPU 上，结合 pthread 线程池模型，可以在提高资源利用率的同时显著减少线程管理开销，使多线程加速更加平滑和高效。

任务被切分成多个线程并行执行，理论加速比接近核心数 P ，实际通常受到内存带宽、线程管理开销和同步延迟限制。

(三) MPI 多进程

MPI (Message Passing Interface) 用于在分布式或多节点环境中进行多进程并行。每个进程拥有独立地址空间，通过显式消息传递（`MPI_Send`/`MPI_Recv`）交换数据。MPI 常用于大规模集群计算：

- **领域划分**: 将全局数据切分到各个进程, 本地计算后再做必要的边界通信。
- **集体通信**: 广播 (MPI_Bcast)、归约 (MPI_Reduce) 等高效实现。

MPI 的加速取决于计算通信比, 若计算量远大于通信量, 能接近线性扩展。

(四) GPU 加速计算

GPU (Graphics Processing Unit) 拥有成百上千的流处理器, 采用 SIMT (Single Instruction, Multiple Threads) 模型:

- 将问题划分为大量细粒度线程 (几十到上万), 在多个线程块 (block) 和线程 (thread) 中并行执行同一核函数 (kernel)。
- 通过在主机 (CPU) 与设备 (GPU) 间拷贝数据, 再在 GPU 上并行计算, 最后将结果传回。

得益于超高的并行度和内存带宽, GPU 在矩阵运算、向量运算、图像处理等高度并行任务中常能实现数十倍乃至上百倍的加速, 但需要考虑内存传输开销和线程调度开销。

四、 实验设计探究

在我们本次实验中, 由于不同硬件环境的情况不同, 我们更需要对比的是每一个方法实现对同环境下朴素算法的对比, 注意由于硬件需要暖机, 我们在正式统计前进行暖机; 正式测试时, 采用多次重复取平均值的方法以消除偶发抖动。

(一) PQ 类方法 +SIMD

在之前的实验中我们对比分析了 PQ 类算法和 SIMD 的组合对 ANN 算法的加速效果, 在本次报告中我们将沿用这个实验的实验框架。

(二) IVF/HNSW 类方法 + 多线程

对于 IVF/HNSW 方法, 我们沿用之前的测试框架对其进行分析, 重点在于之前的实验结果的纠错与分析重置

(三) 多进程优化算法

在多进程优化算法中, 我们将探究分块并行的 HNSW 算法的结果并进行分析, 将数据拆分为 P 个独立分区, 并在每个分区上构建 HNSW:

1. **数据分块**: 将全集随机或基于启发式 (如聚类、哈希划分) 分为 P 个部分, 每部分大小约为 N/P 。
2. **并行构建**: 启动 P 个 MPI 进程, 各自在其分区上独立构建 HNSW 索引。
3. **并行查询**: 查询时, 各进程在本地 HNSW 上并行搜索 Top- k , 并将结果与距离返回主进程。
4. **全局合并**: 主进程在收到 P 份候选后维护全局小顶堆, 得到最终的 Top- k 结果。

(四) GPU 加速查询间并行

对于 GPU 的查询加速,我们重点在于探讨 GPU 的吞吐能力对算法的并行效果,我们探讨如何使用 GPU 加速距离计算,加速 TOP-K 选择,对于 IVF 算法,我们探讨 GPU 的加速能力与不同分组方式的关系,针对其设计实验。

(五) 探究: kdtree 算法与 LSH 算法并行化尝试

在我们的本次实验中我们拓展了 kdtree 算法和 LSH 算法,我们分别对其与朴素算法作加速测试,并探究使用 simd+openmp 对其加速的实验方式。³

(六) 探究: 杂交方法效果

在目前的所有实现中,我们主要设计的杂交方法是 IVF+PQ 与 IVF+HNSW 两种方式,我们将根据之前的实验结果,对比其不同的性能实现。

(七) 探究: 混合并行尝试

混合并行尝试我们主要涉及 HNSW 对于不同边数图的处理和 MPI 多级流水线算法,对于 HNSW 算法,我们将设置不同的并行阈值探讨混合并行的实现与优势,对于流水线算法,我们将补充分析其效率实现及其表现原因。

五、实验结果可视化分析

(一) PQ 类方法 +SIMD

1. 实验结果

我们在本地测试得到的主要实验结果如表1,具体可视化分析如下图5:

2. 召回率与延迟的权衡分析

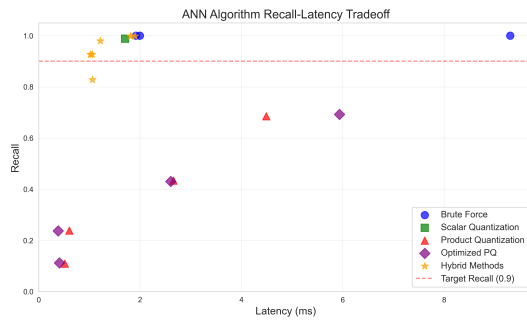
图5a展示了不同 ANN 算法在召回率和查询延迟之间的权衡关系。⁴从图中可以清晰地看出:

- 暴力搜索算法(特别是标量实现)虽然提供了精确召回,但其查询延迟也最高。
- 标量量化(SQ)技术维持了接近暴力搜索的高召回率,同时显著降低了延迟。
- 乘积量化方法(PQ和OPQ)随着子空间数量(M)的增加,召回率逐渐提高,但查询延迟也相应增加。
- 混合搜索方法位于图的左上区域,表明它们在保持高召回率的同时实现了低延迟,是性能与精度平衡的最佳选择。

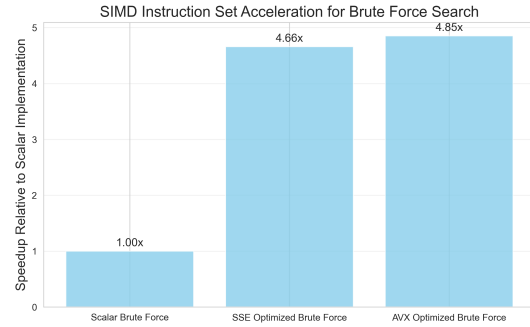
对于并行算法而言,算法的 recall 和 latency 的 tradeoff 关系比较明显。混合搜索(PQ16+精确重排序)和 PQ16+R100 实现了超过 92% 的召回率,同时延迟仅为 1.02 毫秒,表现出最佳的综合性能平衡,体现出先近似查找再进行精确查找方法的优越性。

³我们本想使用 GPU 对 LSH 这个比较偏向矩阵的算法进行优化,但我们发现集群 GPU 服务器中 nvcc 与 gcc 并不适配而个人服务器余额已经结束,于是我们采用的优化方法为 simd+openmp

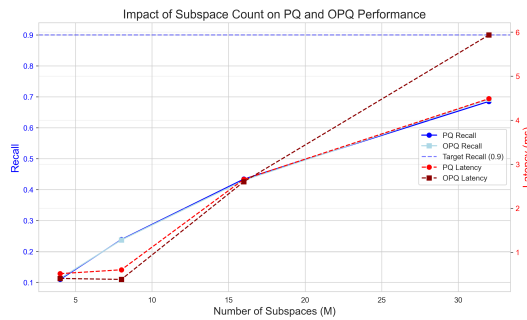
⁴由于此处测试数据组数较少,我们选择以散点的形式呈现这一张图,后续的数据我们将进行连线



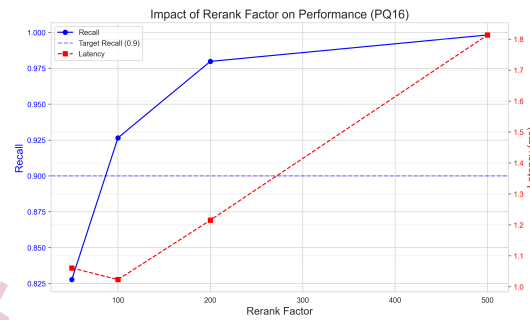
(a) 召回率-延迟权衡分析



(b) SIMD 加速效果



(c) 子空间数量影响



(d) 重排序因子影响

图 5: 不同 ANN 算法实验结果子图合集

另一个比较需要注意的是, SQ 算法在本数据集中表现优异可能是因为对于归一化的数据集而言, 找到其上下限并进行分为 256 个标度本身颗粒度就很小了, 这导致 SQ 的表现优异。

3. SIMD 指令集加速效果

图5b展示了 SIMD 指令集对暴力搜索算法的加速效果。从图中可以清晰看出: 这些加速在保持近似 100% 召回率的前提下实现, 证明了 SIMD 向量化对距离计算的高效性。

4. 子空间数量对 PQ 和 OPQ 性能的影响

图5c展示了子空间数量 (M) 对 PQ 和 OPQ 性能的影响。从图中可以观察到:

- 随着子空间数量的增加, PQ 和 OPQ 的召回率逐渐提高, 但查询延迟也显著增加。
- 在低子空间数量 ($M=4,8$) 时, OPQ 的召回率略高于 PQ, 且延迟更低, 显示了优化变换矩阵带来的益处。
- 当 M 值较大时 ($M=16,32$), 两种方法的召回率差异减小, 但 OPQ 的延迟略高于 PQ。
- 对于追求高召回率的应用场景, $M=32$ 是较好的选择; 而对于追求低延迟的场景, $M=4$ 或 $M=8$ 可能更为合适。

分析: 为什么重排序能够相对单纯的 PQ 提升那么显著?

这个问题存在两方面的解释, 其一是在我们进行 kmeans 聚类时没有使用内积距离实现, 导致后续使用内积举例作并行优化时的使用的聚类本身具有一定的近似性, 而重排序消除了这种近

表 1: ANN 算法综合性能对比 (最优值标粗)

算法	召回率	延迟 (毫秒)	加速比
标量暴力搜索	1.0000	9.30	1.00×
SSE 优化暴力搜索	1.0000	2.00	4.66×
AVX 优化暴力搜索	1.0000	1.92	4.85×
标量量化 (SQ)	0.9878	1.70	5.46×
乘积量化 (PQ) M=4	0.1095	0.52	18.04×
乘积量化 (PQ) M=8	0.2385	0.60	15.41×
乘积量化 (PQ) M=16	0.4340	2.66	3.50×
乘积量化 (PQ) M=32	0.6851	4.49	2.07×
优化乘积量化 (OPQ) M=4	0.1124	0.41	22.86×
优化乘积量化 (OPQ) M=8	0.2372	0.39	24.16×
优化乘积量化 (OPQ) M=16	0.4303	2.60	3.57×
优化乘积量化 (OPQ) M=32	0.6920	5.94	1.57×
混合搜索 (PQ16+ 精确重排序)	0.9265	1.02	9.09×
混合搜索 (PQ32+ 精确重排序)	0.9982	1.81	5.13×
混合搜索 (OPQ16+ 精确重排序)	0.9269	1.05	8.82×
混合搜索 (OPQ32+ 精确重排序)	0.9982	1.90	4.90×
PQ16+R50	0.8277	1.06	8.77×
PQ16+R100	0.9265	1.02	9.09×
PQ16+R200	0.9798	1.21	7.66×
PQ16+R500	0.9982	1.81	5.13×

似性, 从而使得精度提升很快。其二, 在计算过程中 PQ 使用计算时 Float32 运算的误差存在累积, 导致 PQ/OPQ 算法的正确率下降, 而重排序计算时使用原始向量精度会略微高一些。

5. 重排序因子对性能的影响

图5d分析了重排序因子对混合搜索方法性能的影响。从图中可以得出以下结论:

- 随着重排序因子的增加, 召回率呈现明显的上升趋势, 但查询延迟也相应增加。
- 重排序因子为 100 时, 实现了 92.65% 的高召回率, 同时维持了较低的查询延迟 (1.02 毫秒), 是精度和性能的良好平衡点。
- 当重排序因子增加到 500 时, 召回率进一步提高到 99.82%, 接近暴力搜索的精度, 但延迟增加到 1.81 毫秒, 已经和暴力搜索没有区别甚至超越了暴力搜索的时间消耗。

出于平衡考虑, 对于大多数实际应用, 重排序因子在 100-200 之间可能是最佳选择, 既保证了高召回率, 又维持了较低的延迟。

(二) IVF/HNSW 类方法 + 多线程

1. 朴素 IVF 算法多线程

我们在 4 线程下利用 openmp 和 pthread 对朴素的 ivf 算法进行了优化，其大致实验结果如下图6所示：

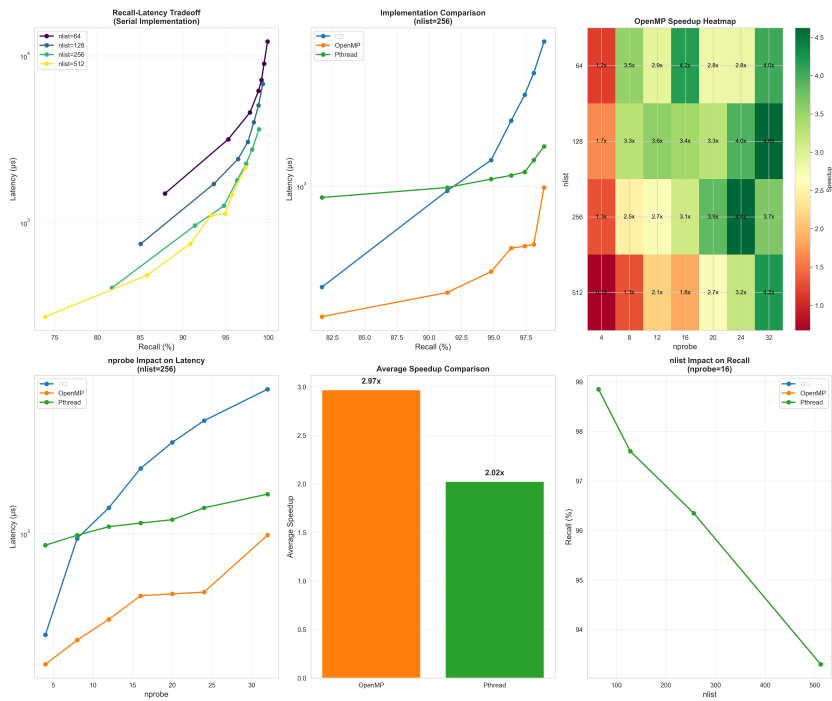


图 6: 朴素 ivf 多线程

可以比较明显的发现，openmp 和 pthread 对 ivf 算法均有不错的优化效果，尤其加入线程池后 pthread 能够出现较好的优化，虽然由于其调度开销较大不如系统实现了指令级别优化的 openmp，但也成功对 ivf 进行了成功的优化。

2. HNSW 算法及其多线程

类似的我们使用 openmp 对 HNSW 算法的算法内部进行了并行加速，具体来讲我们对其 KNN 过程进行了多线程优化，其结果如7所示。

但与 IVF 算法的普遍加速相比，HNSW 算法由于我们查询的规模相对较小即使是 openmp 算法中调度开销也超过了多线程带来的收益，于是我们尝试使用混合并行的方式动态决定是否使用多进程，尽量使得算法健壮，后续我们将专门概述这个内容。

(三) 多进程优化算法

1. 分片 HNSW 算法

在这个实验中，我们采用了四种分块策略进行尝试，分别是随机分块、轮询分块 (即取模分块)、哈希分块和 KMEANS 分块将数据分为同样数量的块 (这里是 4) 到进程中进行测试，将测试序列中的向量分到不同的进程中进行 HNSW 查询，其中随机分块与轮询分块不考虑向量内部情况，哈希与 KMEANS 尝试利用向量中的一部分信息让每一块中的向量分布更有规律性从而加速查询。

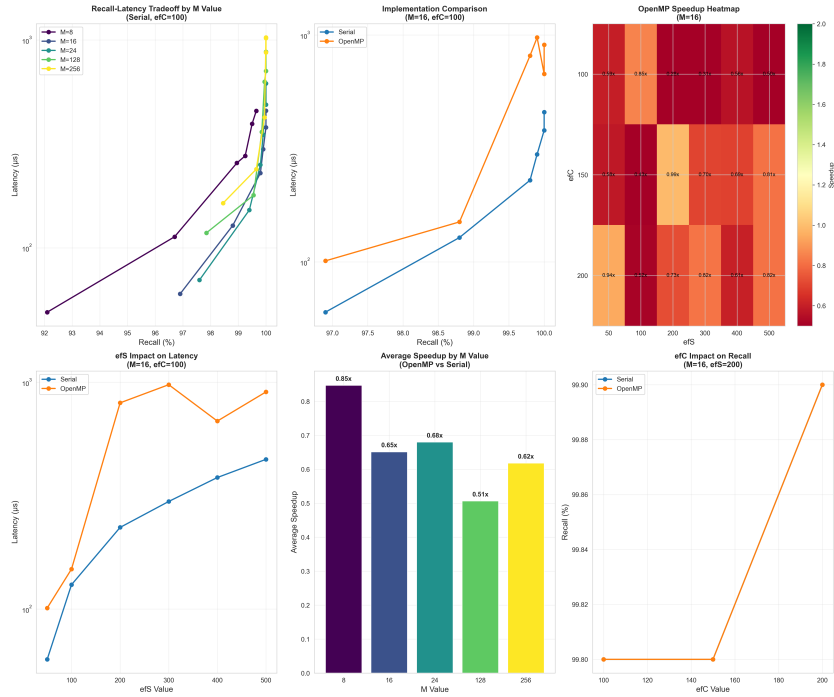
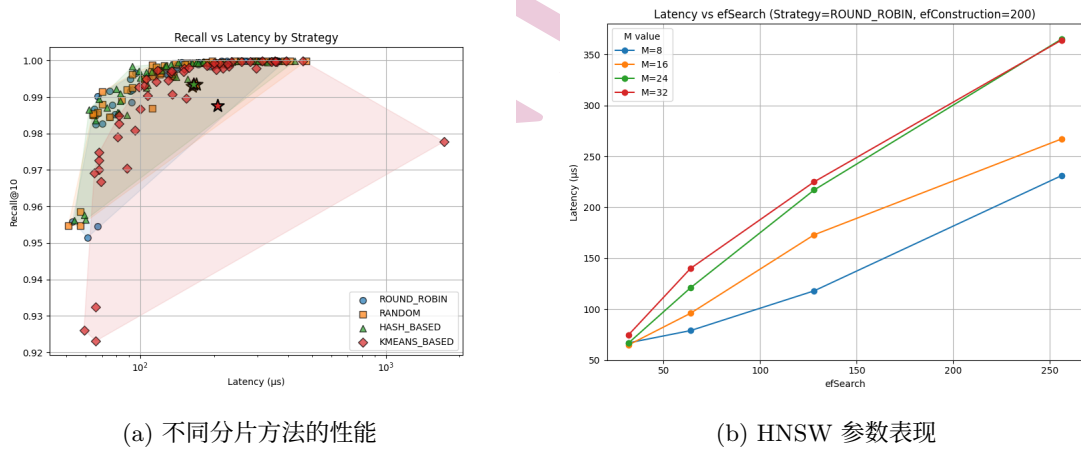


图 7: HNSW 算法 openmp 优化

我们得到在不同参数条件下暖机后的实验结果数据并对其进行分析,用以下可视化方式表示实验结果图8a展示了在不同分片方法下的性能表现,可以看到由于数据分布由于其一般性,我们对其进行的人为聚类反而造成了一定的性能损失,但相比于不分片的情况性能有所提升。



(a) 不同分片方法的性能

(b) HNSW 参数表现

图 8: 分片方法与参数表现对比

2. 进程数对实验结果的影响分析

我们以朴素 IVF 算法 (不含 PQ 与 HNSW) 在不同进程下的并行表现为例分析多进程进程数对算法效率的影响。实验采用 1 进程、2 进程和 4 进程三种配置,每种配置下均使用 4 个 OpenMP 线程,在相同的硬件环境和数据集上进行对比测试。

整体性能表现分析 实验结果显示,不同进程数配置下的性能表现存在显著差异:

表 2: 不同 MPI 进程数配置的性能对比

进程数	平均延迟 (s)	平均构建时间 (ms)	平均召回率	算法效率
1	532.1	779.8	0.7849	1.4749
2	399.0	241.5	0.7849	1.9669
4	640.9	560.8	0.7849	1.2246

整体如下图9所示:

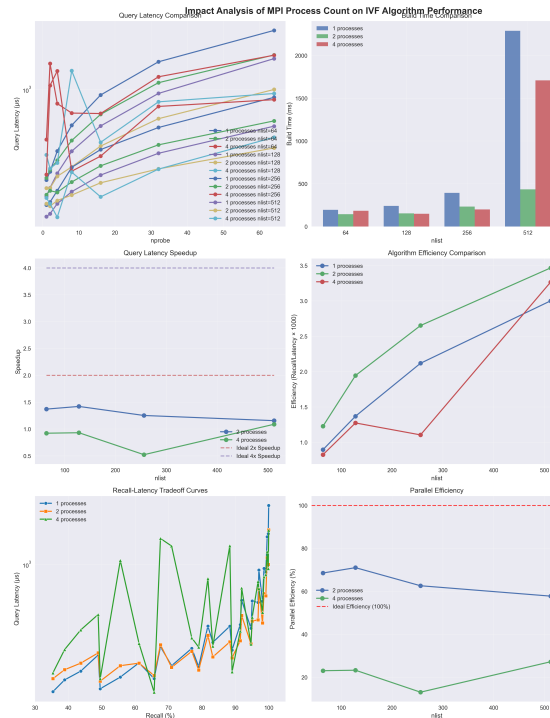


图 9: IVF 算法不同进程数下的性能表现对比

从整体性能表现可以看出:

- **2 进程配置表现最优:** 在查询延迟、构建时间和算法效率三个关键指标上均优于其他配置;
- **4 进程配置性能下降:** 相比单进程配置, 4 进程在查询延迟和算法效率上都出现了一定倒退;
- **召回率保持稳定:** 所有配置下的召回率完全一致, 验证了算法正确性。

扩展性规律总结 通过对不同进程数配置的分析, 我们发现了 IVF 算法 MPI 并行化的扩展性规律:

1. **存在最优进程数:** 并非进程数越多性能越好, 需要根据具体的硬件和数据特征确定最优配置;
2. **构建与查询阶段特性不同:** 构建阶段更适合并行化, 查询阶段对通信延迟更敏感;
3. **数据规模影响扩展性:** 不同的 nlist 设置下, 最优进程数可能有所不同;
4. **硬件架构制约:** 当前的硬件配置下, 2 进程是通信开销与并行收益的最佳平衡点。

(四) GPU 加速查询间并行

1. GPU 批次加速方案加速比

我们把不同的加速比的实验分析结果展示如下图10

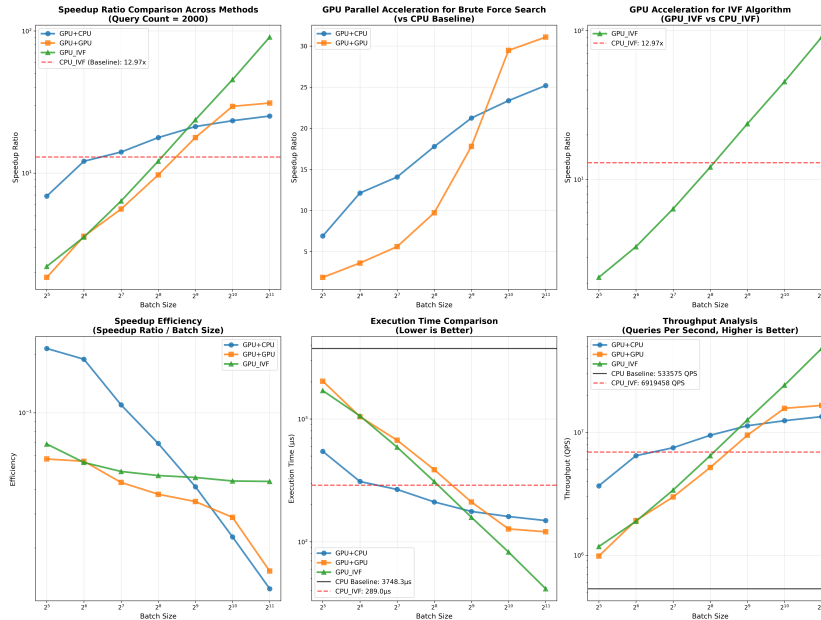


图 10: GPU 批次加速对比实验结果

相比 CPU 实现 GPU 的加速效应 从可视化结果可以观察到:

- **稳定性对比:** GPU+CPU 表现出更好的线性增长特性, 而 GPU+GPU 呈现指数型性能增长, 后续我们将深入分析这个问题
- **峰值性能:** GPU+GPU 在大批量处理时达到 31.08 倍的峰值加速比, 超越 GPU+CPU 的 25.19 倍

GPU 批次对 IVF 算法的加速分析 GPU_IVF 算法结合了 IVF 索引的智能性和 GPU 并行计算的强大能力, 实现了所有测试方案中的最优性能:

- **性能交叉点:** GPU_IVF 在批大小 256 附近 (12.12 倍) 开始接近 CPU_IVF 的性能 (12.97 倍)
- **超越阈值:** 从批大小 512 开始, GPU_IVF 显著超越 CPU_IVF, 最终实现 6.97 倍的相对加速
- **极致性能:** 在批大小约为 2K 时, GPU_IVF 达到 90.41 倍的极致加速比, 相对于 CPU 基准提升 98.9%
- **吞吐量优势:** 峰值吞吐量达到 48,270,677 QPS, 是 CPU_IVF 的 7 倍, CPU 基准的 90 倍

讨论: batchsize 与 query 数量分析 GPU 的吞吐优势 实验结果清晰展示了批大小作为 GPU 性能关键因子的重要作用:

- **效率递减规律:** 随着批大小增加, 单位批量的加速效率呈递减趋势
- **大批量优势:** 从图中可以很明显的发现, GPU 算法基本都在吞吐量较大时开始“发力”, 实现效率的提升, 这很明显反映了 GPU 并行计算的特性。

2. CPU 选择 TOP-K 与 GPU 实现 TOP-K 加速对比

我们对比了两种方案的性能: (1) GPU 计算矩阵乘法后使用 CPU 进行 TOP-K 选择 (简称 GPU+CPU 方案) 和 (2) 完全在 GPU 上进行矩阵运算和 TOP-K 选择 (简称 GPU+GPU 方案)。实验通过不同批大小 (Batch Size) 和查询数量 (Query Count) 的组合进行了全面测试, 以分析两种方案在各种工作负载下的表现, 其表现大致如下图11所示

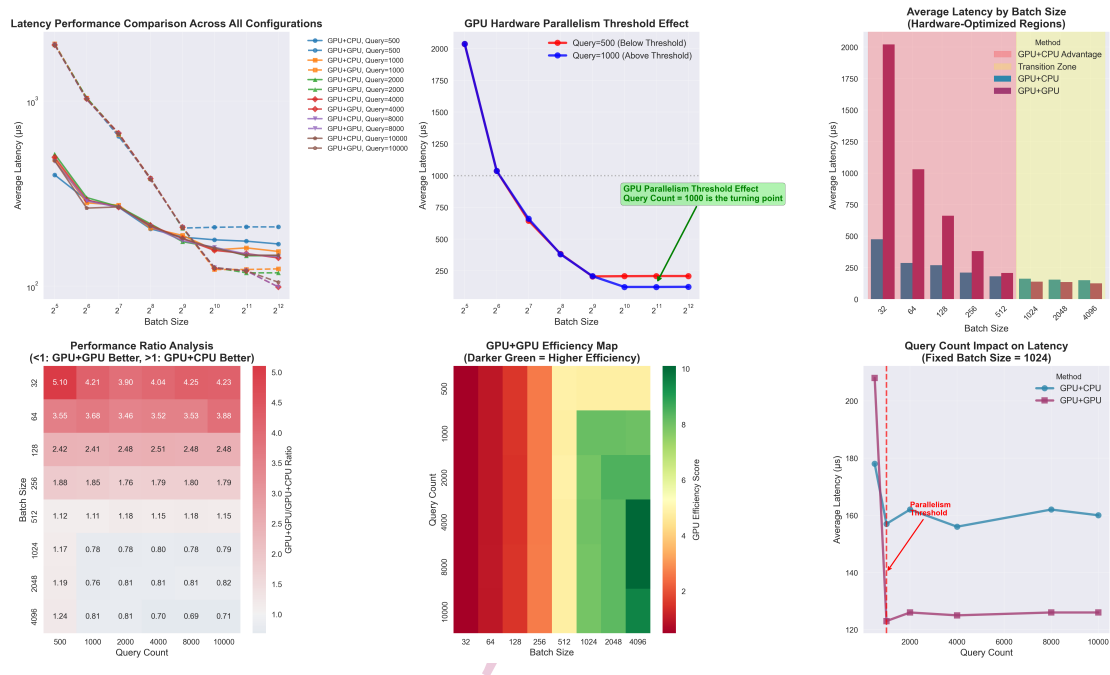


图 11: 不同 TOP-K 选择策略效果对比

召回率分析 从实验数据来看, 两种方案在召回准确性方面不存在准确性问题, 这表明 GPU 进行 TOP-K 选择不会影响准确搜索的结果质量。

延迟分析 从实验结果可以比较明显的观察到, 由于 GPU 在批次较小时由于通信开销明显大于计算开销故 GPU+GPU 方案表现远远不如 GPU+CPU, 而后续的实现中我们发现 GPU+CPU 更早触及边际效应, GPU+GPU 方案方案表现则快速优化直至比 GPU+CPU 良好, 从上述折线图可以非常明显地看到这个现象。

加速比分析 我们可以通过计算两种方案的加速比 (GPU+GPU/GPU+CPU) 来量化性能差距:

- 在小批量处理时 (Batch Size ≤ 256), GPU+CPU 方案具有显著优势, 延迟比通常大于 3。
- 随着批大小增加到 512, 延迟比降至 1.2 左右。

- 当批大小达到 1024 及以上时，延迟比约为 0.7-0.8，表明 GPU+GPU 方案开始展现性能优势。

综上，我们的实验对比了在向量检索任务中，两种不同的 TOP-K 选择方案的实验效果，而我们发现再不同的批次处理条件下对 TOP-K 的选择情况，这与我们在第二次实验中探讨的混合并行的思想不谋而合，在后续的工作中我们也将探讨混合并行的可能性。

3. 不同分组方式对比

我们对 9 种不同的查询分组策略进行全面对比分析。实验涵盖了批大小从 64 到 1024 的范围，通过簇重叠率与延迟的关系揭示各策略的性能特征和适用场景实验结果可视化如下图12所示

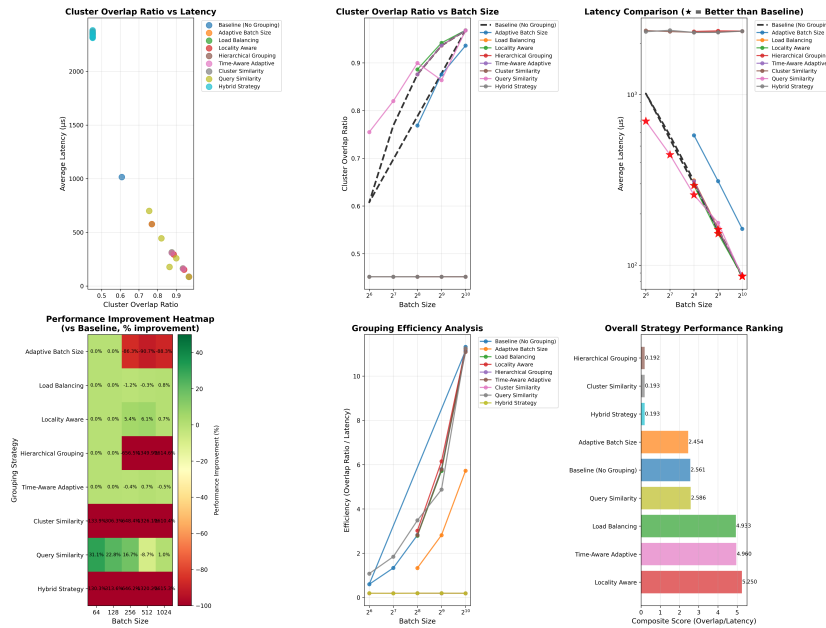


图 12: 不同分组模式对比，其中基线表现特殊标出，以区分不同分组策略的优劣

簇重叠率与延迟关系分析 簇重叠率是衡量分组策略效果的关键指标，它反映了同一批次内查询访问相同簇的程度。理论上，较高的簇重叠率意味着更好的内存访问局部性和缓存利用率，从而带来更低的延迟。

- 高重叠率区间 (>0.9):** 局部性感知、时间感知自适应、负载均衡等策略表现优异，延迟控制在 86-294 s 范围内
- 中重叠率区间 (0.6-0.9):** 查询相似性分组策略在此区间表现稳定，延迟介于 258-699 s
- 低重叠率区间 (<0.45):** 簇相似性、混合策略和分层分组策略重叠率较低，延迟显著增加至 2300 s 以上

批大小对分组策略的影响 batchsize 会影响分组策略性能，不同策略在不同批大小下的表现差异显著：

- 基准策略:** 表现出典型的单调增长模式，重叠率从 0.608 (批大小 64) 增长至 0.968 (批大小 1024)，批次增大会导致重叠增大，这是自然且符合直觉的。

- **局部性感知策略**：在所有批大小下均优于基准，重叠率最高达 0.968，大批大小时收敛于基准水平
- **查询相似性策略**：在中等批大小（256-512）时达到最优重叠率 0.900，但在大批大小时收敛于基准水平
- **问题策略**：簇相似性、混合策略和分层分组在所有批大小下均维持较低的重叠率（0.452）

策略性能提升分析 通过性能提升热力图分析，可以量化各策略相对于基准的改进程度。实验结果显示，不同策略在各批大小下的表现存在显著差异，其表现与簇重叠率的高低直接相关：

优秀表现策略性能表：

策略	批大小	性能提升 (%)	簇重叠率	适用场景
查询相似性	64	31.1	0.755	小批量优势
	128	22.8	0.820	中小批量
	256	16.7	0.900	中等批量
局部性感知	256	5.4	0.886	中等批量
	512	6.1	0.942	大批量优势
	1024	0.7	0.968	大批量稳定
负载均衡	1024	0.8	0.968	大批量均衡
时间感知自适应	512	0.7	0.937	中大批量

表 3: 优秀策略性能提升详细数据

查询相似性策略在小到中等批大小时表现卓越，最高可获得 31.1% 的性能提升；局部性感知策略在中到大批大小时表现稳定优异。相反，簇相似性、混合策略和分层分组策略由于过度细分导致 GPU 并行度不足，性能表现不佳甚至不如不进行特殊处理的组。

（五）探究：kdtree 算法与 LSH 算法并行化尝试

我们尝试了对 kdtree 算法进行以降低一定 recall 为代价的近似 kdtree 算法,包括限制 kdtree 的回溯次数等方法,对于 LSH 算法,由于其本身更加依赖向量的维数,在 96 维的条件下表现相对糟糕,我们增加了其表数,对其进行了优化分析。

我们在 DEEP100K 数据集（96 维，100k 序列，测试查询 2000 条）上，对多种 KDTree 与 LSH 算法及其并行/近似变种进行了构建与查询性能测试，结果汇总如表 4 和表 5 所示。

分析与讨论

- **KDTree 近似 vs 精确**：将 KDTree 回溯次数限制以实现 $\approx 90\%$ 召回时，查询延迟锐减至 $\sim 631\mu s$ （加速 $5.85\times$ ），但召回率仅 $\sim 40.9\%$ 。结合 SIMD 后的并行版本性能基本一致，说明近似剪枝效果主导性能提升。
- **LSH 参数优化**：标准 LSH 在高维环境下召回率低（21.0%），但查询极快（ $70.5\mu s$ ， $52\times$ 加速）。为冲刺 90% 召回，我们增加哈希表数（表数）并做参数优化，建索引时间大幅增长至 14280ms，查询延迟提升至 $1635\mu s$ ，QPS 仍可达 608。SIMD 并行可再提升约 $1.5\times$ 。
- **并行化效果对比**：SIMD 并行对 Flat Search 和 LSH 提升显著（ $4.5\sim 52\times$ ），对 KDTree 近似版本则主要受限于剪枝开销，与串行版差别不大；KDTree 森林（Ensemble）并行固有并行度有限，整体加速不足。

表 4: 索引构建时间对比 (ms)

算法	构建时间 (ms)
Flat Search (无索引)	0
KDTree CPU	306
LSH CPU	1915
KDTree Approx (90% recall)	307
LSH Improved (90% recall)	14280
KDTree SIMD Parallel	337
LSH SIMD Parallel	14615
KDTree Ensemble Parallel (12 棵树)	1911
Flat Search SIMD Parallel	0

表 5: 纯查询性能对比

算法	Recall	Latency (us)	QPS	相对加速比
Flat Search	0.99995	3695.4	269.6	1.00×
Flat Search SIMD Parallel	0.99995	815.9	1213.6	4.53×
KDTree CPU	0.99695	10289.7	97.1	0.36×
KDTree Approx (90% recall)	0.40885	631.3	1564.9	5.85×
KDTree SIMD Parallel	0.42485	636.4	1560.1	5.81×
KDTree Ensemble Parallel	1.00000	4579.6	218.0	0.81×
LSH CPU	0.21010	70.5	13513.5	52.42×
LSH Improved (90% recall)	0.88710	1635.4	608.8	2.26×
LSH SIMD Parallel	0.85375	1077.2	924.2	3.43×

- **索引构建与查询权衡:** KDTree 及其近似和 SIMD 版本构建开销小 (~300ms), 适合对实时索引有要求的场景; 而 LSH 为提高召回需付出较重的构建成本, 适合批量离线构建后多次查询。

1. 参数分析

我们对 KDTree 混合优化和 LSH SIMD 并行算法的关键参数进行了敏感性评估, 结果如图 14 和图 13 所示。

关键发现

- **KDTree 混合优化:**
 - 树数量增大可提升召回 (99.9%→100%), 但带来线性延迟增长;
 - 最佳配置: 5 棵树 + 3000 节点, 实现 $\geq 99.9\%$ 召回与 2.08× 加速。
- **LSH SIMD 并行:**
 - 哈希表数从 60→150, 召回率由 73.4% 提升至 89.4%, 加速比由 2.57× 降至 1.37×;
 - 推荐: 60-90 表, 可在 2.5× 以上加速与 ~80% 召回间取得良好平衡。

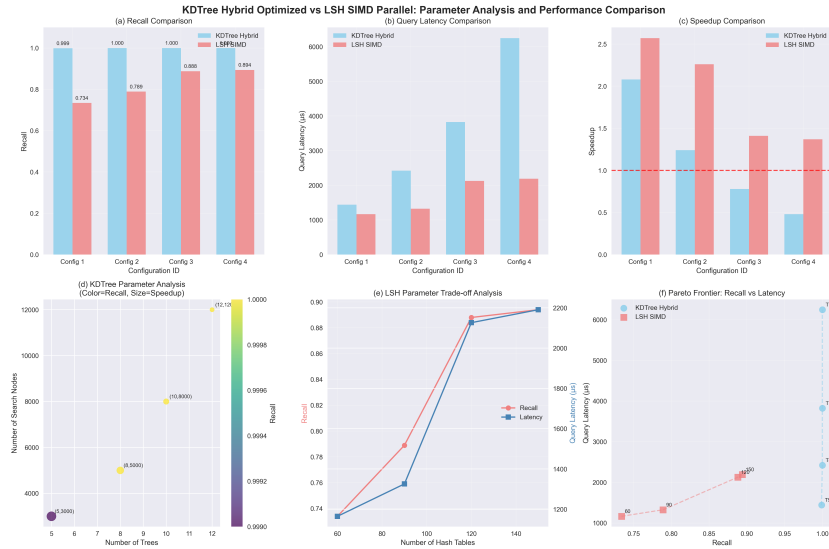


图 13: KDTree 混合优化：树数量 vs. 搜索节点数

(六) 探究：杂交方法效果

1. PQ 与 IVF 混合的两种方法对比分析

我们以 4 进程带重排序的固定参数的结果为例进行分析，我们可以得到如下结果如图15所示：通过上面的实验，我们可以对比分析如下：

IVF-PQ 混合策略对比分析 IVF 与 PQ 的混合使用主要存在两种策略，其核心区别在于**码本训练的先后顺序、码本的作用域（全局 vs. 簇内）**以及由此带来的**编码精度、存储开销和查询效率**上的差异。下面分别从几个维度进行分析，我们总结两者的区别如下表6

表 6: PQ→IVF 与 IVF→PQ 对比

特性	PQ→IVF	IVF→PQ
码本训练方式	全局一次性训练	每簇独立训练 (n_{list} 次)
量化误差	较大	较小
存储开销	较低 (1 套码本)	较高 (n_{list} 套码本)
查询查表次数	1 次	n_{probe} 次
查询速度	快	略慢
检索准确率	一般	更高
训练时间	较短	较长

2. HNSW 与 IVF 算法混合方法分析

我们以 4 进程环境下的 IVF+HNSW 混合索引为例，从算法两个阶段——IVF 粗筛与 HNSW 精排——分别讨论它们对最终召回率和查询延迟的影响，借助参数的变化，我们可以分析两种混合的算法的作用，下图16是其直观化表示。

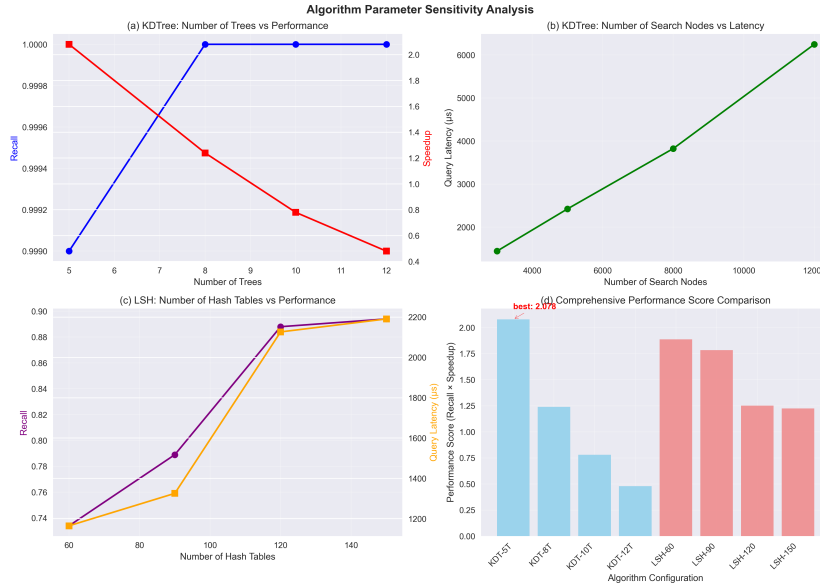


图 14: LSH SIMD 并行：哈希表数 vs. 召回率/加速比

1. IVF 粗筛阶段的作用 IVF 将数据分为 $n_{list} = 64$ 个簇，每次查询仅探测 n_{probe} 个最相近簇，因而将 HNSW 搜索空间从全量 100k 向量缩减到约

$$\frac{n_{probe}}{n_{list}} \times 100\,000$$

大幅降低后续图搜索成本。实验中随着 n_{probe} 从 2 增加到 16，查询延迟由 $\sim 50\mu s$ 缓慢上升至 $\sim 200\mu s$ ，而召回率则从 0.65 快速跃升至 0.95 以上。可见，IVF 粗筛对召回率的提升具有“断崖式”效果——当 n_{probe} 较小时，大量近邻落在未探测簇中；当 $n_{probe} \gtrsim 8$ 时，主要近邻均被包含，召回迅速逼近 0.9。

2. HNSW 精排阶段的作用 在每个被选中的粗簇内部，HNSW 用 beam search（搜索深度参数 $efSearch$ ）对局部候选作精细排序。实验数据显示：

- 当 $efSearch$ 从 32 提升到 128 时，召回率在低 n_{probe} 情况下有明显补漏效果，可再提升约 2-3%；
- 进一步增大至 256，则召回提升边际递减，而延迟却呈近线性增长。

这说明：在 IVF 已完成大部分近邻筛选后，HNSW 主要负责“补漏”——在少量簇内深入搜索，提高整体召回率。

3. 整体权衡与参数调整 混合方式将两者优势结合：IVF 快速排除绝大多数远邻，HNSW 在小范围内做高质量搜索。基于上述实验结果，我们给出以下参数推荐：

- **低延迟场景** ($latency < 100\mu s$): $n_{probe} = 4-8$, $efSearch = 32$, 可达召回 0.80-0.92；
- **高召回场景** ($recall \geq 0.95$): $n_{probe} = 8-16$, $efSearch = 64-128$, 召回可达 0.95-0.97, 延迟约 120-300 μs ；
- **极致召回** ($recall \geq 0.98$): 需 $n_{probe} \geq 16$ 且 $efSearch \geq 128$, 延迟 300-500 μs , 可根据资源预算选择最优组合。

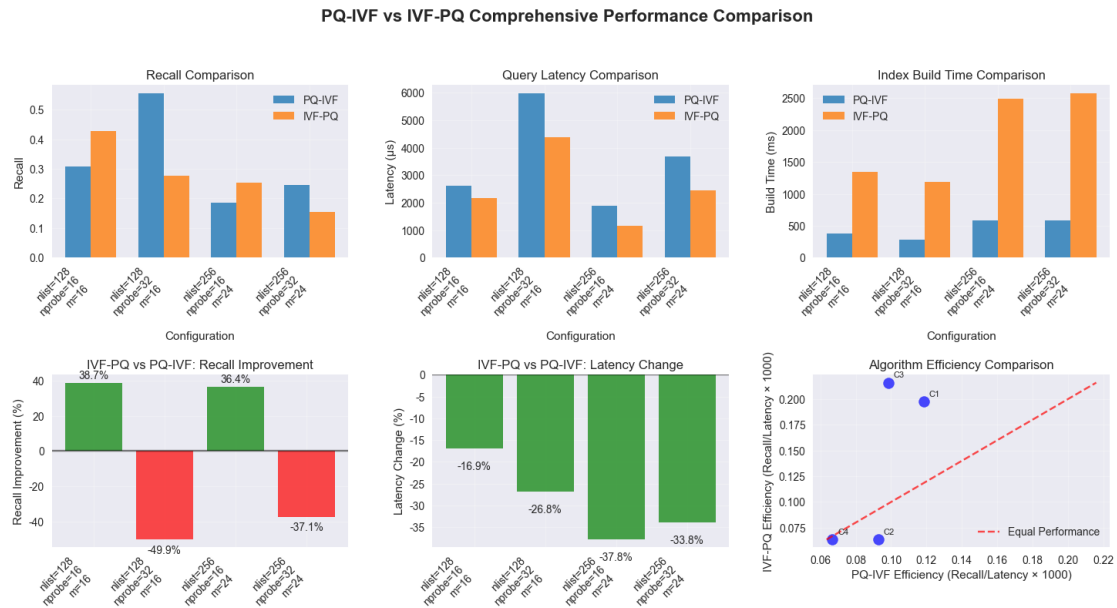


图 15: 两种策略的对比

综上所述，我们可以知道，IVF 阶段决定了召回的“下限”——通过调整 n_{probe} 快速控制召回率门槛；HNSW 阶段则负责在这一基础上做“微调”，进一步提升召回率但带来额外延迟。混合索引通过将粗排与精排有机结合，能够在召回与延迟之间实现更优折中，特别适合需要在百微秒级内实现高召回的在线检索场景。

(七) 探究：混合并行尝试

1. HNSW 算法混合并行尝试

由于我们对 HNSW 算法进行了基于 OpenMP 的并行优化，但得到的结果却是性能下降。为探究其原因，我们使用 perf 工具，对比了不同并行阈值设置下的硬件性能指标。阈值含义如下：

- **纯串行**：阈值设为 0，所有节点均使用串行搜索。
- **纯并行 (T=0)**：阈值远小于最小邻居数，所有节点均使用并行搜索。
- **T=16**：仅当邻居列表长度 ≥ 16 时使用并行，否则串行。
- **T=32**：仅当邻居列表长度 ≥ 32 时使用并行，否则串行。

所有配置下平均召回率均为 0.916604。以下表格7汇总了各方案的平均延迟及 perf 采集到的关键指标：

表 7: 不同并行阈值下 HNSW 搜索的 perf 性能对比

方案	Latency (us)	cycles	instructions	cache-misses	branch-misses
纯串行	192.393	1.14×10^9	1.02×10^9	5.77×10^6	3.58×10^6
纯并行 (T=0)	987.211	30.22×10^9	69.40×10^9	22.20×10^6	10.11×10^6
混合并行 (T=16)	833.880	23.46×10^9	63.56×10^9	10.27×10^6	5.35×10^6
混合并行 (T=32)	188.865	1.15×10^9	1.02×10^9	5.16×10^6	3.42×10^6

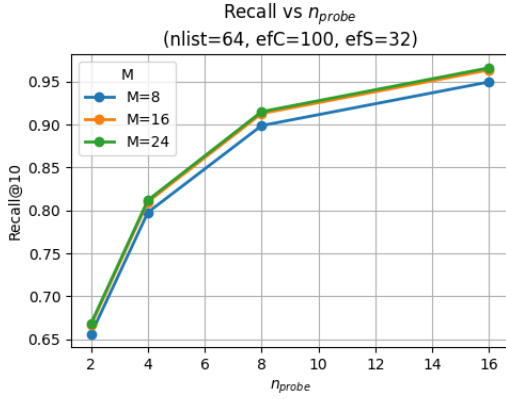
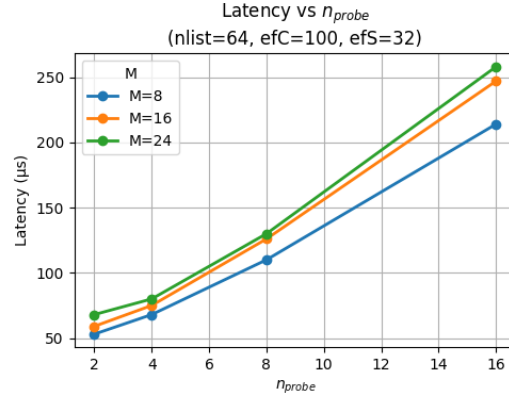
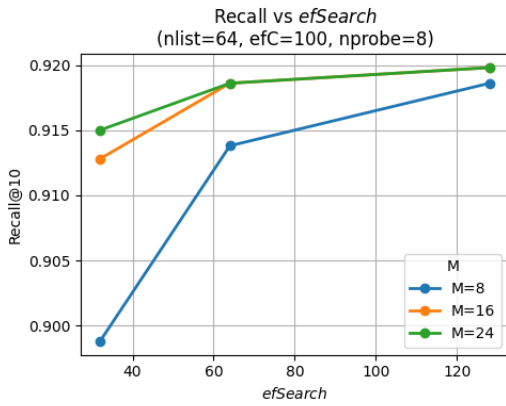
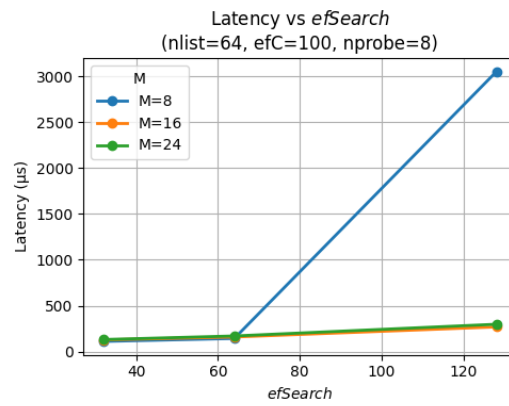
(a) 召回率 vs. n_{probe} ($efSearch = 32$)(b) 延迟 vs. n_{probe} ($efSearch = 32$)(c) 召回率 vs. $efSearch$ ($n_{probe} = 8$)(d) 延迟 vs. $efSearch$ ($n_{probe} = 8$)

图 16: IVF+HNSW 混合索引不同参数下的性能折线图。(a)(b) 中展示了随着 n_{probe} 增大, 召回率的“断崖式”提升与延迟的线性增长; (c)(d) 则展示了在中等粗筛强度下, HNSW 精排深度 $efSearch$ 对召回率补漏与延迟代价的折中。

从表中可以看出:

1. 纯并行较纯串行增加了大量的 CPU 周期与指令数 (cycles $\uparrow 26\times$ 、instructions $\uparrow 68\times$), 并未带来延迟下降, 反而因线程管理、原子操作和缓存抖动导致 latency 飙升。
2. 在 $T=16$ 时, 部分小列表仍走并行, 缓存未命中与分支失误减少不到一半, 延迟仍高达 833 us。
3. 当阈值提升至 $T=32$ 后, 几乎所有节点都退回串行分支, 其性能指标几乎与纯串行重合 (latency 192 us, cycles 1.14×10^9 等), 说明并行分支几乎未被触发。

综合分析, 过于细粒度的并行 (阈值过低) 反而因同步和原子开销而大幅拖慢, 合理的阈值应基于邻居列表长度分布动态设定, 仅对“足够大”的列表启用并行, 才能在串行效率和并行加速之间找到平衡。

2. 流水线并行结果分析

我们以两线程下多级并行 IVF 算法与普通 MPI-IVF 在相同参数 (2 个 MPI 进程 $\times 4$ 线程, AVX2 加速)、不同 nlist (64、128、256、512) 和 nprobe (1-64) 配置下进行对比。

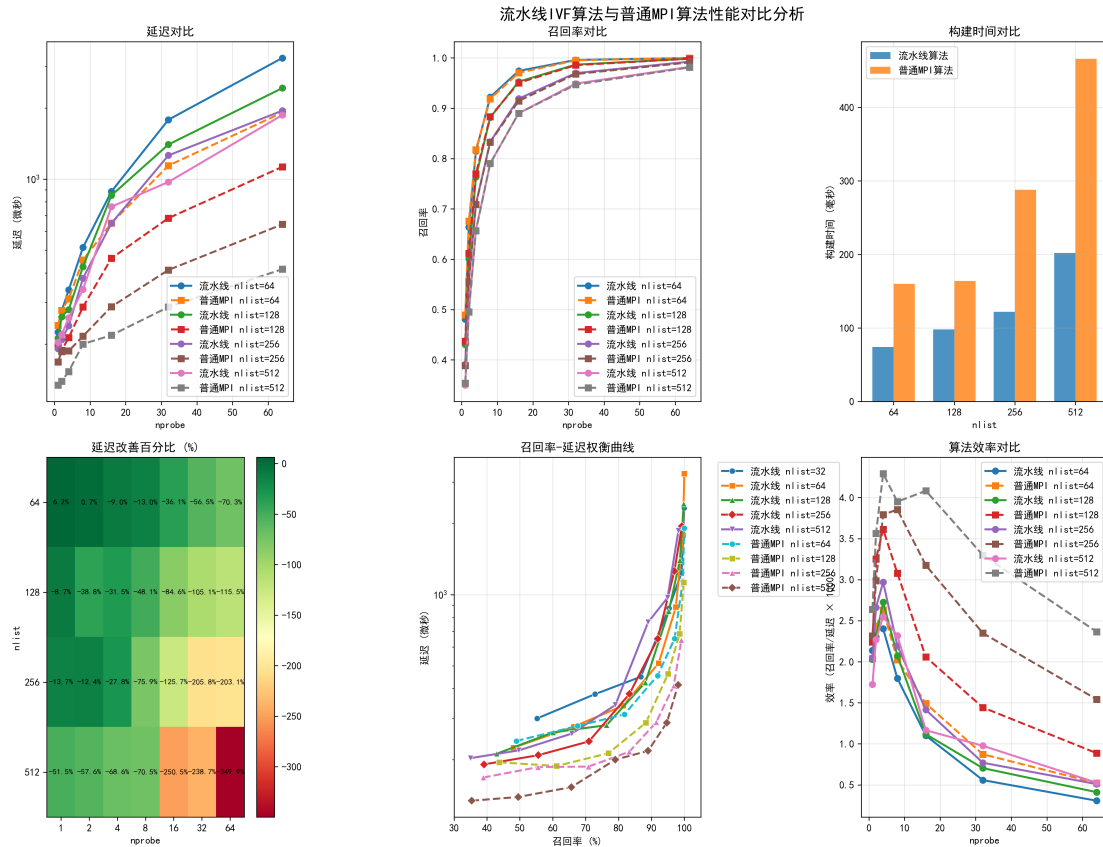


图 17: 流水线多级并行与朴素 MPI-IVF 对比

查询延迟 流水线方案平均延迟比 MPI-IVF 高 84.35%，仅在极少数配置下见微弱改进 (6.25%)；最差时增加达 349.88%。

召回率 两者召回率曲线重合度 >99.9%，说明流水线改造未影响检索质量。

索引构建时间 值得注意的是，索引构建阶段的性能对比显示流水线算法具有明显优势：

表 8: 索引构建时间对比 (ms)

nlist	流水线	MPI-IVF	提升
64	74	160	53.8%
128	98	164	40.2%
256	122	288	57.6%
512	202	466	56.7%
平均	124.0	269.5	54.0%

原因分析 流水线方案的延迟劣势主要源于：

- 线程管理与同步开销增加；
- 多线程并发导致的内存带宽与缓存竞争；
- 四级流水线各阶段负载不均衡，瓶颈积累；

- 虽用非阻塞 MPI，但通信与同步实际仍产生额外延迟。

六、 综合分析

在本报告中，我们针对多种近似最近邻（ANN）算法的并行优化方法，从算法层面与硬件层面对比了它们在召回率、延迟和实现复杂度上的差异，并归纳出以下几点跨实验的发现：

1. **准确率—延迟权衡**：如图 6 和表 1 所示，PQ 和 OPQ 随子空间数 M 增大，召回率逐步提升，但延迟明显增加；而混合方法（PQ+ 精确重排序）在保持高召回率（ $>92\%$ ）的同时，将延迟压至约 1 ms，展现了最佳的性能—精度平衡
2. **并行粒度控制**：对 HNSW 算法的并行阈值实验表明，过于细粒度的并行（阈值 $T = 0$ ）会因线程管理及原子操作开销导致性能大幅下降；合理地仅对较长邻居列表（如 $T \geq 32$ ）启用并行，可在串行效率与并行加速之间找到最佳平衡。
3. **流水线与构建／查询分离**：将 IVF 查询流程设计为“粗排 $\rightarrow$ 异步簇内搜索 $\rightarrow$ 非阻塞汇总”四级流水线，虽然查询延迟平均高出纯 MPI-IVF 84.4%，但索引构建时间平均缩短 54.0%，适合对离线构建时间敏感而对单次查询延迟容忍的场景。
4. **多进程 MPI 扩展性**：MPI-IVF 在进程数为 2 时达到最佳：平均查询延迟 399 μs ，较单进程（532 μs ）提升 25.1%；4 进程反而因通信竞争而退化至 640 μs ，说明并行度提升需要平衡计算与通信开销。
5. **GPU 批量吞吐优势**：GPU 在批大小较小时（ ≤ 256 ）由于传输开销，CPU+GPU 方案（CPU 做 Top-K）优于全 GPU 方案；而当批大小 ≥ 1024 时，全 GPU 方案加速比超越 1 $\times$ ，并在 IVF 索引上实现最高 90.4 $\times$ 峰值加速，适合大批量离线查询。
6. **查询分组策略**：基于查询相似性或局部感知的分组，可提高簇重叠率至 0.9 以上，从而带来最高 31.1% 的延迟优化；而过度细分（簇相似性、分层分组）则因并行度不足反而降低性能。

综上所述，不同并行技术和 ANN 算法之间存在显著的“硬件—算法—参数”三维度交互效应，最佳配置需结合实际查询规模、硬件架构与准确率需求动态调整。

七、 总结

本次并行优化实验报告基于 DEEP100K 数据集，系统性地探索了 SIMD、OpenMP/pthread、多进程 MPI、GPU 以及多种 ANN 索引方法的融合应用，并得出以下的总结：

- **算法压缩与重排序**：PQ + 精确重排序在召回率和延迟之间实现优异折中，是高召回低延迟场景的首选。
- **并行阈值动态策略**：HNSW 的并行加速需基于邻居列表长度动态设定阈值，避免过细粒度带来同步和缓存竞争的负面影响。
- **流水线优化权衡**：多级流水线显著降低索引构建时间，适用于对构建效率有刚需的批量检索任务；对低延迟实时查询可优先选择纯 MPI 或 GPU 并行方案。
- **MPI 进程数选择**：在当前硬件环境下，2 进程 + OpenMP 多线程为 MPI-IVF 提供最佳扩展性，避免过多进程引发的通信瓶颈。

- **GPU 批量吞吐与 Top-K 策略:** 大批量查询 (≥ 1024) 应尽量使用全 GPU 方案以发挥吞吐优势; 小批量查询 (≤ 256) 可在 GPU 后使用 CPU 完成 Top-K, 以减少数据传输开销。
- **分组策略优化:** 基于查询相似性的批次分组能有效提升缓存命中与内存局部性, 实现显著延迟优化; 需避免过度细化分组以保障并行度。

行文至此, 本学期并行程序设计课程围绕 ANN 搜索选题的学习与实践也已圆满结束。从最初对并行计算模型与 CPU 架构的入门认识, 到基于 SIMD、OpenMP、MPI 和 GPU 的多维度性能实验, 我们系统地掌握了并行编程的基本思路与优化策略。过程中, 我们不仅实现了对 Flat Search、KDTree、LSH 等经典算法的串行版本, 更通过剪枝近似、参数调优、流水线设计以及深度并行化尝试, 不断权衡召回率、延迟、吞吐与构建开销, 验证了“算法—硬件—并行模式”三者协同的重要性。数据可视化与定量分析帮助我们直观地对比各方案优劣, 形成了面向不同应用场景的性能选型指导。在课程的学习不仅提升了我们在 C++、CUDA 与并行调试方面的实践能力, 也深化了对高性能计算与大规模近似检索原理的理解。最后衷心感谢老师和各位助教的讲解, 让我在并行计算方面有了更为深刻的理解。

MINU